commands.

In order to use variables, make assertions, and do all the things you'd expect to do in a test, Cypress allows you to chain methods to the `cy` command and also allows you to use `as` to hold on to values for later use.

So in our test, the line

```
cy.get(".concert").first().as("concert")
```

allows us to hold on to the value returned by `first()` using the alias `concert`, such that subsequent Cypress commands can use `cy.get("@concert")` to access the same DOM element. Later lines of code chain the method `should` into the `cy` command so that the assertions take place inside the Cypress asynchronous commands.

Although we don't use it in this test, you can chain the method `then` to any Cypress command to execute arbitrary code inside Cypress, like this:

```
cy.get(".concert").first().then(element => {
  // do whatever we want in here...
}
```

From Cypress's perspective, making each command asynchronous gives Cypress full control over how each test executes. When testing JavaScript, the fact that DOM elements are often changing, appearing, or disappearing makes the tests very complicated to run, and an extremely common problem is that the test and the timing of the DOM changes don't quite line up, leaving tests that intermittently fail for no reason.

Cypress attempts to bypass those flaky timing errors by guaranteeing that each command is only executed after the previous command has fully completed. The first line (after login) of our test is `cy.visit("/")`. The second line, `cy.get("#favorite-section").as("favorites")` only begins execution after the `visit` command has completed. Furthermore, if the `cy.get("#favorite-section")` command does not initially find any matching elements in the DOM, Cypress will automatically wait—by default, up to four seconds—for the element to show up. Only once it shows up does